JOBSHEET VI

Mata Kuliah: Praktikum Algoritma Dan Struktur Data

SORTING (BUBBLE, SELECTION, DAN INSERTION SORT)

Dosen Pengampu:

Mungki Astiningrum, S.T., M.Kom.

Disusun oleh:

Ahmad Rafid Riqkullah

254107020078

T1 – 1G

D-IV Teknik Informatika

POLITEKNIK NEGERI MALANG

Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur 65141

TAHUN 2025-2026

5.2 Praktikum 1 - Mengimplementasikan Sorting menggunakan object

5.2.1 Langkah Praktikum 1

a. **SORTING – BUBBLE SORT**

- **Sorting**

```
public class Sorting03 {
    int[] data;
    int jumData;

    // Konstruktor
    public Sorting03(int Data[], int jmlDat) {
        jumData = jmlDat;
        data = new int[jmlDat];
        for (int i = 0; i < jumData; i++) {
            data[i] = Data[i];
        }
    }

    // Method bubbleSort menggunakan algoritma Bubble Sort (ASC)
    void bubbleSort() {
        int temp = 0;
        for (int i = 0; i < jumData - 1; i++) {
            for (int j = 1; j < jumData - i; j++) {
                if (data[j - 1] > data[j]) {
                    temp = data[j];
                    data[j] = data[j - 1];
                    data[j - 1] = temp;
                }
            }
        }
    }

    // Method untuk menampilkan isi array
    void tampil() {
        for (int i = 0; i < jumData; i++) {
            System.out.print(data[i] + " ");
        }
        System.out.println();
    }

}
```

- **SortingMain**

```
public class SortingMain03 {
   public static void main(String[] args) {
      // --- 1. Pengujian BUBBLE SORT ---
      int a[] = { 20, 10, 2, 7, 12 };
      Sorting03 dataurut1 = new Sorting03(a, a.length);

      System.out.println("Data awal 1");
      dataurut1.tampil();

      dataurut1.bubbleSort();

      System.out.println("Data sudah diurutkan dengan BUBBLE SORT
(ASC)");
      dataurut1.tampil();
         }
}
```

5.2.2 Verifikasi Hasil Percobaan

```
Data awal 1
20 10 2 7 12
Data sudah diurutkan dengan BUBBLE SORT (ASC)
2 7 10 12 20
-----------------------------------------------
```

b. SORTING – SELECTION SORT

- **Sorting**

```
public class Sorting03 {
   int[] data;
   int jumData;

   // Konstruktor
   public Sorting03(int Data[], int jmlDat) {
      jumData = jmlDat;
      data = new int[jmlDat];
      for (int i = 0; i < jumData; i++) {
         data[i] = Data[i];
      }
   }

   // Method bubbleSort menggunakan algoritma Bubble Sort (ASC)
   void bubbleSort() {
      int temp = 0;
      for (int i = 0; i < jumData - 1; i++) {
```

```
            for (int j = 1; j < jumData - i; j++) {
                if (data[j - 1] > data[j]) {
                    temp = data[j];
                    data[j] = data[j - 1];
                    data[j - 1] = temp;
                }
            }
        }
    }

    // Method untuk menampilkan isi array
    void tampil() {
        for (int i = 0; i < jumData; i++) {
            System.out.print(data[i] + " ");
        }
        System.out.println();
    }

    // Method Selection Sort
    void SelectionSort() {
        for (int i = 0; i < jumData - 1; i++) {
            int min = i;
            for (int j = i + 1; j < jumData; j++) {
                if (data[j] < data[min]) {
                    min = j;
                }
            }
            int temp = data[i];
            data[i] = data[min];
            data[min] = temp;
        }
    }

}
```

- **SortingMain**

```
public class SortingMain03 {
    public static void main(String[] args) {
        // --- 1. Pengujian BUBBLE SORT ---
        int a[] = { 20, 10, 2, 7, 12 };
        Sorting03 dataurut1 = new Sorting03(a, a.length);

        System.out.println("Data awal 1");
```

```
        dataurut1.tampil();

        dataurut1.bubbleSort();

        System.out.println("Data sudah diurutkan dengan BUBBLE SORT
(ASC)");
        dataurut1.tampil();

        System.out.println("-----------------------------------------------");

        // --- 2. Pengujian SELECTION SORT ---
        int b[] = { 30, 20, 2, 8, 14 };
        Sorting03 dataurut2 = new Sorting03(b, b.length);

        System.out.println("Data awal 2");
        dataurut2.tampil();

        dataurut2.SelectionSort();

        System.out.println("Data sudah diurutkan dengan SELECTION
SORT (ASC)");
        dataurut2.tampil();
            }
}
```

5.2.3 Verifikasi Hasil Percobaan

```
Data awal 1
20 10 2 7 12
Data sudah diurutkan dengan BUBBLE SORT (ASC)
2 7 10 12 20
-----------------------------------------------
Data awal 2
30 20 2 8 14
Data sudah diurutkan dengan SELECTION SORT (ASC)
2 8 14 20 30
-----------------------------------------------
```

c. SORTING – INSERTION SORT

- **Sorting**

```
public class Sorting03 {
    int[] data;
    int jumData;

    // Konstruktor
    public Sorting03(int Data[], int jmlDat) {
        jumData = jmlDat;
```

```
    data = new int[jmlDat];
    for (int i = 0; i < jumData; i++) {
        data[i] = Data[i];
    }
}

// Method bubbleSort menggunakan algoritma Bubble Sort (ASC)
void bubbleSort() {
    int temp = 0;
    for (int i = 0; i < jumData - 1; i++) {
        for (int j = 1; j < jumData - i; j++) {
            if (data[j - 1] > data[j]) {
                temp = data[j];
                data[j] = data[j - 1];
                data[j - 1] = temp;
            }
        }
    }
}

// Method untuk menampilkan isi array
void tampil() {
    for (int i = 0; i < jumData; i++) {
        System.out.print(data[i] + " ");
    }
    System.out.println();
}

// Method Selection Sort (Baru ditambahkan)
void SelectionSort() {
    for (int i = 0; i < jumData - 1; i++) {
        int min = i;
        for (int j = i + 1; j < jumData; j++) {
            if (data[j] < data[min]) {
                min = j;
            }
        }
        int temp = data[i];
        data[i] = data[min];
        data[min] = temp;
    }
}
```

```
    // Method Insertion Sort
    void insertionSort() {
        for (int i = 1; i <= data.length - 1; i++) {
            int temp = data[i];
            int j = i - 1;
            while (j >= 0 && data[j] > temp) {
                data[j + 1] = data[j];
                j--;
            }
            data[j + 1] = temp;
        }
    }
}
```

- **SortingMain**

```
public class SortingMain03 {
    public static void main(String[] args) {
        // --- 1. Pengujian BUBBLE SORT ---
        int a[] = { 20, 10, 2, 7, 12 };
        Sorting03 dataurut1 = new Sorting03(a, a.length);

        System.out.println("Data awal 1");
        dataurut1.tampil();

        dataurut1.bubbleSort();

        System.out.println("Data sudah diurutkan dengan BUBBLE SORT
(ASC)");
        dataurut1.tampil();

        System.out.println("-----------------------------------------------");

        // --- 2. Pengujian SELECTION SORT ---
        int b[] = { 30, 20, 2, 8, 14 };
        Sorting03 dataurut2 = new Sorting03(b, b.length);

        System.out.println("Data awal 2");
        dataurut2.tampil();

        dataurut2.SelectionSort();
```

```
        System.out.println("Data sudah diurutkan dengan SELECTION
SORT (ASC)");
        dataurut2.tampil();

        System.out.println("-----------------------------------------------");

        // --- 3. Pengujian INSERTION SORT ---
        int c[] = { 40, 10, 4, 9, 3 };
        Sorting03 dataurut3 = new Sorting03(c, c.length);

        System.out.println("Data awal 3");
        dataurut3.tampil();

        dataurut3.insertionSort();

        System.out.println("Data sudah diurutkan dengan INSERTION SORT
(ASC)");
        dataurut3.tampil();
    }
}
```

5.2.4 Verifikasi Hasil Percobaan

```
Data awal 1
20 10 2 7 12
Data sudah diurutkan dengan BUBBLE SORT (ASC)
2 7 10 12 20
-----------------------------------------------
Data awal 2
30 20 2 8 14
Data sudah diurutkan dengan SELECTION SORT (ASC)
2 8 14 20 30
-----------------------------------------------
Data awal 3
40 10 4 9 3
Data sudah diurutkan dengan INSERTION SORT (ASC)
3 4 9 10 40
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma &  Struktur Data\Praktik Algo&StrukturData>
```

5.2.5 Pertanyaan!

1. Jelaskan fungsi kode program berikut

```
if (data[j-1]>data[j]){
    temp=data[j];
    data[j]=data[j-1];
    data[j-1]=temp;
}
```

Jawab:
Kode tersebut berfungsi untuk melakukan proses pertukaran nilai (*swap*) antar elemen array. Program akan mengecek apakah nilai pada indeks sebelumnya (data[j-1]) lebih besar dari indeks saat ini (data[j]). Jika bernilai *true*, maka kedua posisi data tersebut ditukar menggunakan bantuan variabel temp agar urutannya menjadi *ascending* (dari kecil ke besar).

2. Tunjukkan kode program yang merupakan algoritma pencarian nilai minimum pada selection sort!
   Jawab:
   ```
   int min = i;
   for (int j = i + 1; j < jumData; j++) {
      if (data[j] < data[min]) {
         min = j;
      }
   }
   ```
3. Pada Insertion sort , jelaskan maksud dari kondisi pada perulangan
   ```
   while (j>=0 && data[j]>temp)
   ```
   Jawab:
   Kondisi tersebut adalah syarat untuk melakukan pergeseran data ke kanan. Perulangan akan terus berjalan jika memenuhi dua syarat berikut:
   - j >= 0 : Memastikan indeks j tidak bernilai negatif agar tidak terjadi *error index out of bounds* (keluar dari batas indeks array).
   - data[j] > temp : Mengecek apakah nilai array pada indeks ke-j lebih besar dari nilai yang sedang disimpan di variabel temp. Jika lebih besar, berarti data tersebut harus digeser ke kanan.

4. Pada Insertion sort, apakah tujuan dari perintah
   ```
   data[j+1]= data[j];
   ```
   Jawab:
   Tujuan dari perintah tersebut adalah untuk menggeser nilai array satu posisi ke kanan. Pergeseran ini dilakukan untuk menyediakan ruang (celah) kosong, sehingga nilai yang ada pada variabel temp nantinya dapat disisipkan (*inserted*) pada posisi urutan yang tepat.

5.3 Praktikum 2- (Sorting Menggunakan Array of Object)

5.3.1 Langkah Praktikum 2 - Mengurutkan Data Mahasiswa Berdasarkan IPK (Bubble Sort)

5.3.2 Langkah-langkah Praktikum 2

- Mahasiswa03

```
package Jobsheet6.Praktikum05;

public class Mahasiswa03 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    Mahasiswa03() {
    }

    Mahasiswa03(String nm, String name, String kls, double ip) {
        nim = nm;
        nama = name;
        kelas = kls;
        ipk = ip;
    }

    void tampilInformasi() {
        System.out.println("Nama: " + nama);
        System.out.println("NIM: " + nim);
        System.out.println("Kelas: " + kelas);
        System.out.println("IPK: " + ipk);
    }
}
```

- MahasiswaBerprestasi03.java

```
package Jobsheet6.Praktikum05;

public class MahasiswaBerprestasi03 {
    Mahasiswa03[] listMhs = new Mahasiswa03[5];
    int idx;

    void tambah(Mahasiswa03 m) {
        if (idx < listMhs.length) {
            listMhs[idx] = m;
            idx++;
        } else {
            System.out.println("Data sudah penuh");
        }
    }

    void tampil() {
```

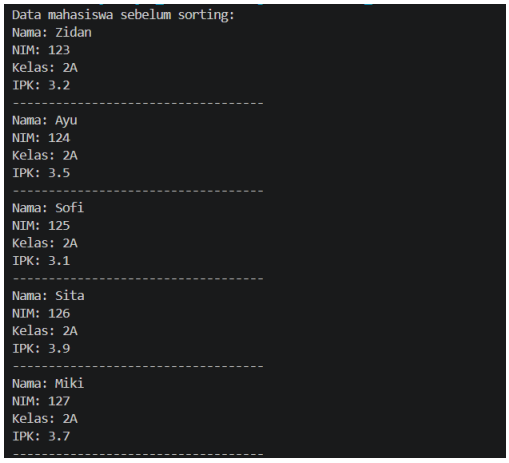
```
        for (Mahasiswa03 m : listMhs) {
            m.tampilInformasi();
            System.out.println("----------------------------------");
        }
    }

    void bubbleSort() {
        for (int i = 0; i < listMhs.length - 1; i++) {
            for (int j = 1; j < listMhs.length - i; j++) {
                if (listMhs[j].ipk > listMhs[j - 1].ipk) {
                    Mahasiswa03 tmp = listMhs[j];
                    listMhs[j] = listMhs[j - 1];
                    listMhs[j - 1] = tmp;
                }
            }
        }
    }
}
```

- MahasiswaDemo03.java

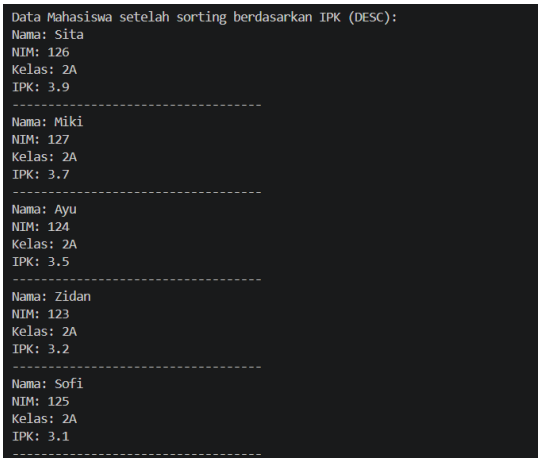
```
package Jobsheet6.Praktikum05;

public class MahasiswaDemo03 {
    public static void main(String[] args) {
        MahasiswaBerprestasi03 list = new MahasiswaBerprestasi03();

        Mahasiswa03 m1 = new Mahasiswa03("123", "Zidan", "2A", 3.2);
        Mahasiswa03 m2 = new Mahasiswa03("124", "Ayu", "2A", 3.5);
        Mahasiswa03 m3 = new Mahasiswa03("125", "Sofi", "2A", 3.1);
        Mahasiswa03 m4 = new Mahasiswa03("126", "Sita", "2A", 3.9);
        Mahasiswa03 m5 = new Mahasiswa03("127", "Miki", "2A", 3.7);

        list.tambah(m1);
        list.tambah(m2);
        list.tambah(m3);
        list.tambah(m4);
        list.tambah(m5);

        System.out.println("Data mahasiswa sebelum sorting: ");
        list.tampil();

        System.out.println("Data Mahasiswa setelah sorting berdasarkan IPK
(DESC): ");
```

```
        list.bubbleSort();
        list.tampil();
    }
}
```

5.3.3 Verifikasi Hasil Percobaan



```
Data mahasiswa sebelum sorting:
Nama: Zidan
NIM: 123
Kelas: 2A
IPK: 3.2
----------------------------------
Nama: Ayu
NIM: 124
Kelas: 2A
IPK: 3.5
----------------------------------
Nama: Sofi
NIM: 125
Kelas: 2A
IPK: 3.1
----------------------------------
Nama: Sita
NIM: 126
Kelas: 2A
IPK: 3.9
----------------------------------
Nama: Miki
NIM: 127
Kelas: 2A
IPK: 3.7
----------------------------------
```



```
Data Mahasiswa setelah sorting berdasarkan IPK (DESC):
Nama: Sita
NIM: 126
Kelas: 2A
IPK: 3.9
----------------------------------
Nama: Miki
NIM: 127
Kelas: 2A
IPK: 3.7
----------------------------------
Nama: Ayu
NIM: 124
Kelas: 2A
IPK: 3.5
----------------------------------
Nama: Zidan
NIM: 123
Kelas: 2A
IPK: 3.2
----------------------------------
Nama: Sofi
NIM: 125
Kelas: 2A
IPK: 3.1
----------------------------------
```

5.3.4 Pertanyaan

1. Perhatikan perulangan di dalam bubbleSort() di bawah ini:

```
for (int i=0; i<listMhs.length-1; i++){
    for (int j=1; j<listMhs.length-i; j++){
```

   a. Mengapa syarat dari perulangan i adalah i<listMhs.length-1?
      Jawab:
      Perulangan luar (for dengan variabel i) berfungsi untuk mengatur berapa banyak tahapan/fase (pass) yang dibutuhkan untuk mengurutkan seluruh array. Syaratnya adalah length - 1 karena pada metode Bubble Sort, setelah kita melakukan perbandingan dan pertukaran hingga elemen kedua terakhir, elemen terakhir otomatis sudah berada di posisi yang benar (sudah terurut). Jadi, jika ada N elemen, kita hanya butuh maksimal N-1 tahapan untuk memastikan seluruh array terurut.

   b. Mengapa syarat dari perulangan j adalah j<listMhs.length-i?
      Jawab:
      Perulangan dalam (for dengan variabel j) berfungsi untuk melakukan perbandingan antar elemen yang bersebelahan dalam satu tahap (pass). Setiap kali satu tahapan (perulangan i) selesai, satu elemen terbesar (atau terkecil, tergantung ascending/descending) pasti sudah "menggelembung" (bubble) dan menempati posisi akhir yang benar. Oleh karena itu, pada tahap berikutnya (saat i

bertambah), kita tidak perlu lagi membandingkan elemen-elemen di posisi akhir yang sudah terurut tersebut. Pengurangan dengan - i membuat area perbandingan menjadi semakin pendek di setiap tahapnya, sehingga program berjalan lebih efisien.

c. Jika banyak data di dalam listMhs adalah 50, maka berapakali perulangan i akan berlangsung? Dan ada berapa Tahap bubble sort yang ditempuh?
Jawab:

Berapa kali perulangan i: Mengikuti syarat i < 50 - 1 (yaitu i < 49), dan i dimulai dari 0. Maka nilai i akan berjalan dari 0, 1 hingga 48. Total perulangan i adalah 49 kali. Tahap bubble sort yang ditempuh: Tahap (pass) yang ditempuh sama dengan jumlah perulangan i, yaitu 49 tahap.

2. Modifikasi program diatas dimana data mahasiswa bersifat dinamis (input dari keyborad) yang terdiri dari nim, nama, kelas, dan ipk!
Jawab:

```
import java.util.Scanner;
public class MahasiswaDemo03 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        MahasiswaBerprestasi03 list = new MahasiswaBerprestasi03();

        System.out.println("=== Masukkan Data Mahasiswa ===");
        for (int i = 0; i < list.listMhs.length; i++) {
            System.out.println("Data Mahasiswa ke-" + (i + 1));

            System.out.print("NIM   : ");
            String nim = sc.nextLine();

            System.out.print("Nama  : ");
            String nama = sc.nextLine();

            System.out.print("Kelas : ");
            String kelas = sc.nextLine();

            System.out.print("IPK   : ");
            double ipk = sc.nextDouble();
            sc.nextLine();

            System.out.println("-------------------------");
```

```
            // Membuat objek Mahasiswa03 baru dan menambahkannya ke list
            Mahasiswa03 m = new Mahasiswa03(nim, nama, kelas, ipk);
            list.tambah(m);
        }

        System.out.println("\n--- Data mahasiswa sebelum sorting ---");
        list.tampil();

        System.out.println("\n--- Data Mahasiswa setelah sorting BUBBLE
SORT berdasarkan IPK (DESC) ---");
        list.bubbleSort();
        list.tampil();

        sc.close();
    }
}
```

5.3.5 Mengurutkan Data Mahasiswa Berdasarkan IPK (Selection Sort)

- Mahasiswa03

```
package Jobsheet6.Praktikum05;

public class Mahasiswa03 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    Mahasiswa03() {
    }

    Mahasiswa03(String nm, String name, String kls, double ip) {
        nim = nm;
        nama = name;
        kelas = kls;
        ipk = ip;
    }

    void tampilInformasi() {
        System.out.println("Nama: " + nama);
        System.out.println("NIM: " + nim);
```

```
        System.out.println("Kelas: " + kelas);
        System.out.println("IPK: " + ipk);
    }
}
```

- MahasiswaBerprestasi03.java

```
public class MahasiswaBerprestasi03 {
    Mahasiswa03[] listMhs = new Mahasiswa03[5];
    int idx;

    void tambah(Mahasiswa03 m) {
        if (idx < listMhs.length) {
            listMhs[idx] = m;
            idx++;
        } else {
            System.out.println("Data sudah penuh");
        }
    }

    void tampil() {
        for (Mahasiswa03 m : listMhs) {
            m.tampilInformasi();
            System.out.println("-----------------------------------");
        }
    }

    void bubbleSort() {
        for (int i = 0; i < listMhs.length - 1; i++) {
            for (int j = 1; j < listMhs.length - i; j++) {
                if (listMhs[j].ipk > listMhs[j - 1].ipk) {
                    Mahasiswa03 tmp = listMhs[j];
                    listMhs[j] = listMhs[j - 1];
                    listMhs[j - 1] = tmp;
                }
            }
        }
    }

    void selectionSort() {
        for (int i = 0; i < listMhs.length - 1; i++) {
            int idxMin = i;
            for (int j = i + 1; j < listMhs.length; j++) {
                if (listMhs[j].ipk < listMhs[idxMin].ipk) {
```

```
                idxMin = j;
            }
        }
        Mahasiswa03 tmp = listMhs[idxMin];
        listMhs[idxMin] = listMhs[i];
        listMhs[i] = tmp;
    }
  }
}
```

- MahasiswaDemo03.java

```
import java.util.Scanner;

public class MahasiswaDemo03 {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    MahasiswaBerprestasi03 list = new MahasiswaBerprestasi03();

    System.out.println("=== Masukkan Data Mahasiswa ===");
    // Karena array di MahasiswaBerprestasi maksimal 5, kita loop 5 kali
    for (int i = 0; i < list.listMhs.length; i++) {
      System.out.println("Data Mahasiswa ke-" + (i + 1));

      System.out.print("NIM   : ");
      String nim = sc.nextLine();

      System.out.print("Nama  : ");
      String nama = sc.nextLine();

      System.out.print("Kelas : ");
      String kelas = sc.nextLine();

      System.out.print("IPK   : ");
      double ipk = sc.nextDouble();
      sc.nextLine();

      System.out.println("-------------------------");

      // Membuat objek Mahasiswa03 baru dan menambahkannya ke list
      Mahasiswa03 m = new Mahasiswa03(nim, nama, kelas, ipk);
      list.tambah(m);
    }
```

```
        System.out.println("\n--- Data mahasiswa sebelum sorting ---");
        list.tampil();

        System.out.println("\n--- Data Mahasiswa setelah sorting BUBBLE
SORT berdasarkan IPK (DESC) ---");
        list.bubbleSort();
        list.tampil();

        System.out.println("\n--- Data Mahasiswa setelah sorting
SELECTION SORT berdasarkan IPK (ASC) ---");
        list.selectionSort();
        list.tampil();
          sc.close();
          }
}
```

5.3.6 Verifikasi Hasil Percobaan

```
=== Masukkan Data Mahasiswa ===
Data Mahasiswa ke-1
NIM   : 123
Nama  : zico
Kelas : 12
IPK   : 3,7
------------------------
Data Mahasiswa ke-2
NIM   : 124
Nama  : zul
Kelas : 12
IPK   : 3,9
------------------------
Data Mahasiswa ke-3
NIM   : 121
Nama  : dapa
Kelas : 12
IPK   : 3,6
------------------------
Data Mahasiswa ke-4
NIM   : 129
Nama  : pan
Kelas : 12
IPK   : 3,5
------------------------
Data Mahasiswa ke-5
NIM   : 128
Nama  : bar
Kelas : 12
IPK   : 3,6
------------------------
```

```
--- Data mahasiswa sebelum sorting ---
Nama: zico
NIM: 123
Kelas: 12
IPK: 3.7
----------------------------------
Nama: zul
NIM: 124
Kelas: 12
IPK: 3.9
----------------------------------
Nama: dapa
NIM: 121
Kelas: 12
IPK: 3.6
----------------------------------
Nama: pan
NIM: 129
Kelas: 12
IPK: 3.5
----------------------------------
Nama: bar
NIM: 128
Kelas: 12
IPK: 3.6
----------------------------------
```

```
--- Data Mahasiswa setelah sorting BUBBLE SORT berdasarkan IPK (DESC) ---
Nama: zul
NIM: 124
Kelas: 12
IPK: 3.9
----------------------------------
Nama: zico
NIM: 123
Kelas: 12
IPK: 3.7
----------------------------------
Nama: dapa
NIM: 121
Kelas: 12
IPK: 3.6
----------------------------------
Nama: bar
NIM: 128
Kelas: 12
IPK: 3.6
----------------------------------
Nama: pan
NIM: 129
Kelas: 12
IPK: 3.5
----------------------------------
```

```
--- Data Mahasiswa setelah sorting SELECTION SORT berdasarkan IPK (ASC) ---
Nama: pan
NIM: 129
Kelas: 12
IPK: 3.5
----------------------------------
Nama: dapa
NIM: 121
Kelas: 12
IPK: 3.6
----------------------------------
Nama: bar
NIM: 128
Kelas: 12
IPK: 3.6
----------------------------------
Nama: zico
NIM: 123
Kelas: 12
IPK: 3.7
----------------------------------
Nama: zul
NIM: 124
Kelas: 12
IPK: 3.9
----------------------------------
```

5.3.7 Pertanyaan

Di dalam method selection sort, terdapat baris program seperti di bawah ini:

```
int idxMin=i;
for (int j=i+1; j<listMhs.length; j++){
    if (listMhs[j].ipk<listMhs[idxMin].ipk){
        idxMin=j;
    }
}
```

Untuk apakah proses tersebut, jelaskan!

Jawab:

Proses kode tersebut berfungsi untuk mencari indeks letak nilai IPK paling kecil (minimum) dari sisa data yang belum terurut.

5.4 Mengurutkan Data Mahasiswa Berdasarkan IPK Menggunakan Insertion Sort

5.4.1 Langkah-langkah Percobaan

- Mahasiswa03

```
package Jobsheet6.Praktikum05;

public class Mahasiswa03 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    Mahasiswa03() {
    }

    Mahasiswa03(String nm, String name, String kls, double ip) {
        nim = nm;
        nama = name;
        kelas = kls;
        ipk = ip;
    }

    void tampilInformasi() {
        System.out.println("Nama: " + nama);
```

```
        System.out.println("NIM: " + nim);
        System.out.println("Kelas: " + kelas);
        System.out.println("IPK: " + ipk);
    }
}
```

- MahasiswaBerprestasi03.java

```
public class MahasiswaBerprestasi03 {
    Mahasiswa03[] listMhs = new Mahasiswa03[5];
    int idx;

    void tambah(Mahasiswa03 m) {
        if (idx < listMhs.length) {
            listMhs[idx] = m;
            idx++;
        } else {
            System.out.println("Data sudah penuh");
        }
    }

    void tampil() {
        for (Mahasiswa03 m : listMhs) {
            m.tampilInformasi();
            System.out.println("-----------------------------------");
        }
    }

    void bubbleSort() {
        for (int i = 0; i < listMhs.length - 1; i++) {
            for (int j = 1; j < listMhs.length - i; j++) {
                if (listMhs[j].ipk > listMhs[j - 1].ipk) {
                    Mahasiswa03 tmp = listMhs[j];
                    listMhs[j] = listMhs[j - 1];
                    listMhs[j - 1] = tmp;
                }
            }
        }
    }

    void selectionSort() {
        for (int i = 0; i < listMhs.length - 1; i++) {
            int idxMin = i;
            for (int j = i + 1; j < listMhs.length; j++) {
```

```
            if (listMhs[j].ipk < listMhs[idxMin].ipk) {
                idxMin = j;
            }
        }
        Mahasiswa03 tmp = listMhs[idxMin];
        listMhs[idxMin] = listMhs[i];
        listMhs[i] = tmp;
    }
}

void insertionSort() {
        for (int i = 1; i < listMhs.length; i++) {
            Mahasiswa03 temp = listMhs[i];
            int j = i;
            while (j > 0 && listMhs[j - 1].ipk > temp.ipk) {
                listMhs[j] = listMhs[j - 1];
                j--;
            }
            listMhs[j] = temp;
        }
    }
}
```

- MahasiswaDemo03.java

```
import java.util.Scanner;

public class MahasiswaDemo03 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        MahasiswaBerprestasi03 list = new MahasiswaBerprestasi03();

        System.out.println("=== Masukkan Data Mahasiswa ===");
        // Karena array di MahasiswaBerprestasi maksimal 5, kita loop 5 kali
        for (int i = 0; i < list.listMhs.length; i++) {
            System.out.println("Data Mahasiswa ke-" + (i + 1));

            System.out.print("NIM   : ");
            String nim = sc.nextLine();

            System.out.print("Nama  : ");
            String nama = sc.nextLine();

            System.out.print("Kelas : ");
```

```
            String kelas = sc.nextLine();

            System.out.print("IPK   : ");
            double ipk = sc.nextDouble();
            sc.nextLine();

            System.out.println("-------------------------");

            // Membuat objek Mahasiswa03 baru dan menambahkannya ke list
            Mahasiswa03 m = new Mahasiswa03(nim, nama, kelas, ipk);
            list.tambah(m);
        }

        System.out.println("\n--- Data mahasiswa sebelum sorting ---");
        list.tampil();

        System.out.println("\n--- Data Mahasiswa setelah sorting BUBBLE
SORT berdasarkan IPK (DESC) ---");
        list.bubbleSort();
        list.tampil();

        System.out.println("\n--- Data Mahasiswa setelah sorting
SELECTION SORT berdasarkan IPK (ASC) ---");
        list.selectionSort();
        list.tampil();

            System.out.println("\n--- Data Mahasiswa setelah sorting
INSERTION SORT berdasarkan IPK (ASC) ---");
        list.insertionSort();
        list.tampil();

            sc.close();
            }
}
```

5.4.2 Verifikasi Hasil Percobaan

```
=== Masukkan Data Mahasiswa ===
Data Mahasiswa ke-1
NIM   : 123
Nama  : zico
Kelas : 12
IPK   : 3,7
------------------------
Data Mahasiswa ke-2
NIM   : 124
Nama  : zul
Kelas : 12
IPK   : 3,9
------------------------
Data Mahasiswa ke-3
NIM   : 121
Nama  : dapa
Kelas : 12
IPK   : 3,6
------------------------
Data Mahasiswa ke-4
NIM   : 129
Nama  : pan
Kelas : 12
IPK   : 3,5
------------------------
Data Mahasiswa ke-5
NIM   : 128
Nama  : bar
Kelas : 12
IPK   : 3,6
------------------------
```

```
--- Data mahasiswa sebelum sorting ---
Nama: zico
NIM: 123
Kelas: 12
IPK: 3.7
----------------------------------
Nama: zul
NIM: 124
Kelas: 12
IPK: 3.9
----------------------------------
Nama: dapa
NIM: 121
Kelas: 12
IPK: 3.6
----------------------------------
Nama: pan
NIM: 129
Kelas: 12
IPK: 3.5
----------------------------------
Nama: bar
NIM: 128
Kelas: 12
IPK: 3.6
----------------------------------
```

```
--- Data Mahasiswa setelah sorting BUBBLE SORT berdasarkan IPK (DESC) ---
Nama: zul
NIM: 124
Kelas: 12
IPK: 3.9
----------------------------------
Nama: zico
NIM: 123
Kelas: 12
IPK: 3.7
----------------------------------
Nama: dapa
NIM: 121
Kelas: 12
IPK: 3.6
----------------------------------
Nama: bar
NIM: 128
Kelas: 12
IPK: 3.6
----------------------------------
Nama: pan
NIM: 129
Kelas: 12
IPK: 3.5
----------------------------------
```

```
--- Data Mahasiswa setelah sorting SELECTION SORT berdasarkan IPK (ASC) ---
Nama: pan
NIM: 129
Kelas: 12
IPK: 3.5
----------------------------------
Nama: dapa
NIM: 121
Kelas: 12
IPK: 3.6
----------------------------------
Nama: bar
NIM: 128
Kelas: 12
IPK: 3.6
----------------------------------
Nama: zico
NIM: 123
Kelas: 12
IPK: 3.7
----------------------------------
Nama: zul
NIM: 124
Kelas: 12
IPK: 3.9
----------------------------------
```

```
--- Data Mahasiswa setelah sorting INSERTION SORT berdasarkan IPK (ASC) ---
Nama: pan
NIM: 129
Kelas: 12
IPK: 3.5
----------------------------------
Nama: dapa
NIM: 121
Kelas: 12
IPK: 3.6
----------------------------------
Nama: bar
NIM: 128
Kelas: 12
IPK: 3.6
----------------------------------
Nama: zico
NIM: 123
Kelas: 12
IPK: 3.7
----------------------------------
Nama: zul
NIM: 124
Kelas: 12
IPK: 3.9
----------------------------------
```

5.4.3 Pertanyaan

Ubahlah fungsi pada InsertionSort sehingga fungsi ini dapat melaksanakan proses sorting dengan cara descending.

Jawab:

- MahasiswaBerprestasi03.java

```
public class MahasiswaBerprestasi03 {
    Mahasiswa03[] listMhs = new Mahasiswa03[5];
    int idx;

    void tambah(Mahasiswa03 m) {
        if (idx < listMhs.length) {
            listMhs[idx] = m;
            idx++;
        } else {
            System.out.println("Data sudah penuh");
        }
    }

    void tampil() {
        for (Mahasiswa03 m : listMhs) {
            m.tampilInformasi();
            System.out.println("-----------------------------------");
        }
    }
    void bubbleSort() {
        for (int i = 0; i < listMhs.length - 1; i++) {
            for (int j = 1; j < listMhs.length - i; j++) {
                if (listMhs[j].ipk > listMhs[j - 1].ipk) {
                    Mahasiswa03 tmp = listMhs[j];
                    listMhs[j] = listMhs[j - 1];
                    listMhs[j - 1] = tmp;
                }
            }
```

```
        }
    }
    void selectionSort() {
        for (int i = 0; i < listMhs.length - 1; i++) {
            int idxMin = i;
            for (int j = i + 1; j < listMhs.length; j++) {
                if (listMhs[j].ipk < listMhs[idxMin].ipk) {
                    idxMin = j;
                }
            }
            Mahasiswa03 tmp = listMhs[idxMin];
            listMhs[idxMin] = listMhs[i];
            listMhs[i] = tmp;
        }
}
void insertionSort() {
        for (int i = 1; i < listMhs.length; i++) {
            Mahasiswa03 temp = listMhs[i];
            int j = i;
            // Baris Ini: tanda '>' diubah menjadi '<'
            while (j > 0 && listMhs[j - 1].ipk < temp.ipk) {
                listMhs[j] = listMhs[j - 1];
                j--;
            }
            listMhs[j] = temp;
        }
    }
}
```

5.5 Latihan Praktikum

Kode Program

- Dosen03

```
package Jobsheet6.Praktikum05;

public class Dosen03 {
    String kode;
    String nama;
    String jenisKelamin;
    int usia;

    // Konstruktor
    public Dosen03(String kd, String name, String jk, int age) {
        kode = kd;
        nama = name;

        if (jk.equalsIgnoreCase("L") || jk.equalsIgnoreCase("Laki-laki") ||
jk.equalsIgnoreCase("Laki laki")) {
            jenisKelamin = "Laki-laki";
        } else if (jk.equalsIgnoreCase("P") ||
jk.equalsIgnoreCase("Perempuan")) {
            jenisKelamin = "Perempuan";
        } else {
            jenisKelamin = jk; // Default jika user mengisi selain L/P
        }

        usia = age;
    }

    // Menampilkan detail satu dosen
    public void tampil() {
        System.out.println("Kode          : " + kode);
        System.out.println("Nama          : " + nama);
        System.out.println("Jenis Kelamin : " + jenisKelamin); // Langsung
panggil variabelnya
        System.out.println("Usia          : " + usia);
        System.out.println("--------------------------------");
    }
}
```

- DataDosen03

```
package Jobsheet6.Praktikum05;

public class DataDosen03 {
    Dosen03[] dataDosen = new Dosen03[10];
    int idx = 0;

    // Menambah data dosen ke array
    public void tambah(Dosen03 dsn) {
        if (idx < dataDosen.length) {
            dataDosen[idx] = dsn;
            idx++;
            System.out.println("Data Dosen berhasil ditambahkan.");
        } else {
            System.out.println("Array Data Dosen sudah penuh!");
        }
    }

    // Menampilkan seluruh isi array dosen
    public void tampil() {
        if (idx == 0) {
            System.out.println("Data Dosen masih kosong.");
            return;
        }
        for (int i = 0; i < idx; i++) {
            dataDosen[i].tampil();
        }
    }

    // Bubble Sort ASC (Termuda ke Tertua)
    public void SortingASC() {
        for (int i = 0; i < idx - 1; i++) {
            for (int j = 1; j < idx - i; j++) {
                if (dataDosen[j - 1].usia > dataDosen[j].usia) {
                    Dosen03 temp = dataDosen[j];
                    dataDosen[j] = dataDosen[j - 1];
                    dataDosen[j - 1] = temp;
                }
            }
        }
    }
```

```
    // Insertion Sort DESC (Tertua ke Termuda)
    public void sortingDSC() {
        for (int i = 1; i < idx; i++) {
            Dosen03 temp = dataDosen[i];
            int j = i;
            while (j > 0 && dataDosen[j - 1].usia < temp.usia) {
                dataDosen[j] = dataDosen[j - 1];
                j--;
            }
            dataDosen[j] = temp;
        }
    }
}
```

- DosenMain03

```
package Jobsheet6.Praktikum05;

import java.util.Scanner;

public class DosenMain03 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        DataDosen03 data = new DataDosen03();
        int menu;

        do {
            System.out.println("\n=== MENU DATA DOSEN ===");
            System.out.println("1. Tambah Data Dosen");
            System.out.println("2. Tampil Seluruh Data Dosen");
            System.out.println("3. Sorting ASC (Termuda ke Tertua - Bubble
Sort)");
            System.out.println("4. Sorting DESC (Tertua ke Termuda -
Insertion Sort)");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu (1-5): ");
            menu = sc.nextInt();
            sc.nextLine();

            switch (menu) {
                case 1:
                    System.out.print("Masukkan Kode Dosen: ");
                    String kode = sc.nextLine();
```

```
                System.out.print("Masukkan Nama Dosen: ");
                String nama = sc.nextLine();

                String jk = "";
                boolean inputValid = false;
                do {
                    System.out.print("Jenis Kelamin (L/P atau Laki-
laki/Perempuan): ");
                    jk = sc.nextLine();

                    // Cek apakah input sesuai dengan yang kita mau
                    if (jk.equalsIgnoreCase("L") || jk.equalsIgnoreCase("Laki-
laki") ||
                        jk.equalsIgnoreCase("P") ||
jk.equalsIgnoreCase("Perempuan")) {
                        inputValid = true; // Jika benar, loop akan berhenti
                    } else {
                        System.out.println("Maaf, input salah! Harap ketik L
atau P saja.\n");
                    }
                } while (!inputValid);

                System.out.print("Masukkan Usia: ");
                int usia = sc.nextInt();
                sc.nextLine();

                Dosen03 dsn = new Dosen03(kode, nama, jk, usia);
                data.tambah(dsn);
                break;

            case 2:
                System.out.println("\n--- Data Seluruh Dosen ---");
                data.tampil();
                break;

            case 3:
                System.out.println("\n--- Hasil Sorting ASC (Termuda ke
Tertua) ---");
                data.SortingASC();
                data.tampil();
                break;
```

```
            case 4:
                System.out.println("\n--- Hasil Sorting DESC (Tertua ke
Termuda) ---");
                data.sortingDSC();
                data.tampil();
                break;

            case 5:
                System.out.println("Program selesai. Terima kasih!");
                break;

            default:
                System.out.println("Pilihan menu tidak valid!");
        }
    } while (menu != 5);

    sc.close();
  }
}
```

Hasil kode program

```
=== MENU DATA DOSEN ===
1. Tambah Data Dosen
2. Tampil Seluruh Data Dosen
3. Sorting ASC (Termuda ke Tertua - Bubble Sort)
4. Sorting DESC (Tertua ke Termuda - Insertion Sort)
5. Keluar
Pilih menu (1-5): 1
Masukkan Kode Dosen: 13
Masukkan Nama Dosen: asd
Jenis Kelamin (L/P atau Laki-laki/Perempuan): l
Masukkan Usia: 33
Data Dosen berhasil ditambahkan.

=== MENU DATA DOSEN ===
1. Tambah Data Dosen
2. Tampil Seluruh Data Dosen
3. Sorting ASC (Termuda ke Tertua - Bubble Sort)
4. Sorting DESC (Tertua ke Termuda - Insertion Sort)
5. Keluar
Pilih menu (1-5): 1
Masukkan Kode Dosen: 14
Masukkan Nama Dosen: ksdm
Jenis Kelamin (L/P atau Laki-laki/Perempuan): p
Masukkan Usia: 40
Data Dosen berhasil ditambahkan.

=== MENU DATA DOSEN ===
1. Tambah Data Dosen
2. Tampil Seluruh Data Dosen
3. Sorting ASC (Termuda ke Tertua - Bubble Sort)
4. Sorting DESC (Tertua ke Termuda - Insertion Sort)
5. Keluar
Pilih menu (1-5): 2
```

```
--- Data Seluruh Dosen ---
Kode          : 13
Nama          : asd
Jenis Kelamin : Laki-laki
Usia          : 33
--------------------------------
Kode          : 14
Nama          : ksdm
Jenis Kelamin : Perempuan
Usia          : 40
--------------------------------

=== MENU DATA DOSEN ===
1. Tambah Data Dosen
2. Tampil Seluruh Data Dosen
3. Sorting ASC (Termuda ke Tertua - Bubble Sort)
4. Sorting DESC (Tertua ke Termuda - Insertion Sort)
5. Keluar
Pilih menu (1-5): 3

--- Hasil Sorting ASC (Termuda ke Tertua) ---
Kode          : 13
Nama          : asd
Jenis Kelamin : Laki-laki
Usia          : 33
--------------------------------
Kode          : 14
Nama          : ksdm
Jenis Kelamin : Perempuan
Usia          : 40
--------------------------------
```

```
=== MENU DATA DOSEN ===
1. Tambah Data Dosen
2. Tampil Seluruh Data Dosen
3. Sorting ASC (Termuda ke Tertua - Bubble Sort)
4. Sorting DESC (Tertua ke Termuda - Insertion Sort)
5. Keluar
Pilih menu (1-5): 4

--- Hasil Sorting DESC (Tertua ke Termuda) ---
Kode          : 14
Nama          : ksdm
Jenis Kelamin : Perempuan
Usia          : 40
--------------------------------
Kode          : 13
Nama          : asd
Jenis Kelamin : Laki-laki
Usia          : 33
--------------------------------

=== MENU DATA DOSEN ===
1. Tambah Data Dosen
2. Tampil Seluruh Data Dosen
3. Sorting ASC (Termuda ke Tertua - Bubble Sort)
4. Sorting DESC (Tertua ke Termuda - Insertion Sort)
5. Keluar
Pilih menu (1-5): 5
Program selesai. Terima kasih!
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma &  Struktur Data\Praktik Algo&StrukturData>
```